Reviewer nNeK

Subject

Review on DynamAuction

Overall assessment

- Positive review (Rating: 5 – Accept).
- Reviewer likes the problem framing.
- Appreciates the benchmark, mathematical formulation, and MPC approach.
- Main concerns are about empirical validation, robustness, and practical applicability.

Response

W1 / Robustness across presets. *""Could the authors provide a small robustness study on at least 2–3 additional built-in presets, such as saturation, recency, or mixed kernels? A reduced comparison among FiPla, Shading, MPC, MPCTimeBinned, and one RL baseline would already be informative.""*

MPC consistently dominates across all four presets, with gains amplified by stronger temporal dependence (Recency: +267%, Saturation: +114% for MPC(H=10)). PPO underperforms FiPla in three of four scenarios despite exhaustive per-scenario hyperparameter search (135-combination grid over $\mathrm{lr} \times \mathrm{ent_coef} \times n_{\mathrm{steps}} \times \lambda_{\mathrm{GAE}}$, 10 M-step retrain), confirming that pure Deep RL approaches struggle in this setting. Shading is scenario-dependent, gaining on Survival but losing on Recency and Mixed.

Table 1: Cross-preset robustness (η=50, 1 000 seeds). Each cell shows mean reward (gain vs. FiPla, the greedy myopic policy with access to the expected-reward oracle). Shading uses the best α per scenario: 0.7 / 0.5 / 0.7 / 0.6 for Survival / Saturation / Recency / Mixed. Shading values updated from 200-seed sweeps to dedicated 1 000-seed runs.

Policy	Survival	Saturation	Recency	Mixed
FiPla	1.706 (—)	1.392 (—)	1.988 (—)	1.766 (—)
Shading (best α)	1.915 (+12.3%)	1.396 (+0.3%)	1.849 (−7.0%)	1.745 (−1.2%)
PPO	1.592 (−6.7%)	1.288 (−7.5%)	1.704 (−14.3%)	1.794 (+1.6%)
MPC(H=10)	2.282 (+33.7%)	2.973 (+113.5%)	7.293 (+266.8%)	3.126 (+77.0%)
MPC(H=10, K=2)	2.352 (+37.9%)	3.095 (+122.3%)	7.896 (+297.1%)	3.108 (+76.0%)

W2 / Sensitivity of MPC to V_0. *""Could the authors add a sensitivity analysis where V_0 is imperfect? For example, evaluate MPC under noisy V_0, under fewer training rollouts for the V_0 estimator, or with a deliberately misspecified valuation head.""*

Table 2 shows mean reward for MPC(H=10) as M, the number of simulation rollouts, varies from 1 to 50 (survival scenario, 1 000 seeds, $\alpha \in \{0.5, 0.6, \ldots, 1.0\}$). Even at M=1 the agent already exceeds FiPla (2.219 > 1.706, +30.1%), and reward is essentially flat from M=5 onward (range 2.305–2.340, within Monte Carlo noise of the planning rollouts). This confirms that the planning gain comes overwhelmingly from the MLP terminal-value head V_0, not from the rollout dynamics themselves: once V_0 provides a reliable bootstrap estimate, additional rollouts yield

negligible improvement. Latency, by contrast, scales approximately linearly with M (11 ms at M=1 to 550 ms at M=50), which directly motivates the choice of a small default (M=10) in our benchmarks.

Table 2: Reward vs. number of planning rollouts M for MPC(H=10) (survival scenario, 1 000 seeds, $\alpha \in \{0.5, 0.6, \ldots, 1.0\}$). FiPla (myopic-optimal baseline): 1.706. Latency scales approximately linearly with M; reward is flat beyond M=1, confirming that planning gain comes from the V_0 terminal-value head.

M (rollouts)	Mean reward	Gain vs. FiPla	Latency (ms)
1	2.219	+30.1%	11.2
5	2.311	+35.4%	55.3
10	2.334	+36.8%	111.1
25	2.305	+35.1%	275.7
50	2.340	+37.2%	550.4

Table 3: MPC mean episode reward (500 seeds) as a function of the number of episodes n_{ep} used to train the V_0 MLP (survival-second-price-exponential, η=50). Performance stabilises around n_{ep}=100.

Policy	n_{ep} (MLP training episodes)					
	10	25	50	100	200	500
MPC(H=10)	1.060	1.715	1.781	2.113	2.156	2.175
MPC(H=25)	1.044	1.842	1.845	2.210	2.309	2.345
MPCTimeBinned(H=10, K=2)	1.110	1.787	1.871	2.148	2.217	2.269
MPCTimeBinned(H=25, K=2)	1.113	1.854	1.983	2.249	2.384	2.439

W3 / RL diagnostic: immediate vs. delayed rewards. *""Could the authors include a small diagnostic experiment with either immediate rewards or oracle-shaped rewards for RL agents? This would help determine whether the RL baselines fail mainly because of planning difficulty, delayed credit assignment, or insufficient training.""*

The immediate-reward variant of the survival kernel reuses the same conversion-probability function f as the delayed kernel:

$$f(\mathbf{u}, \mathbf{x}, \text{ctx}) = \text{clip}\Big(\text{ctx} \cdot (\mathbf{u}^\top \mathbf{w}_u) \cdot (\mathbf{x}^\top \mathbf{w}_x),\ 0,\ 1\Big), \tag{1}$$

where $\mathbf{u}$ is the user embedding, $\mathbf{x}$ the display features, and ctx the campaign context signal. In the delayed kernel κ=0 and conversion times follow an exponential hazard $\lambda(s) = f \cdot \alpha e^{-\alpha s}$; in the immediate variant λ=0 and $\kappa = f$, so $n_\kappa \sim$ Bernoulli(f), i.e. at most one conversion per impression, paid out at bid time.

We retrained PPO from scratch on the immediate kernel in two settings: first with the same hyperparameters used for the delayed scenario (*PPO tuned*), and second with the best configuration from a 124-trial grid search on the immediate kernel (*PPO search*, lr=3×10^{-4}, ent_coef=0.01, n_{steps}=2048, λ_{GAE}=0.9), both for 10 M steps (Table 4). Results for SAC and TD3 are not yet available as retraining takes 9–10 h; we will provide them in the rebuttal discussion.

Retraining on the immediate kernel with the same hyperparameters narrows PPO's gap from -6.7% to -2.4% ($+4.3$ pp), confirming that delayed credit assignment contributes to the underperformance. Crucially, with search-optimised hyperparameters PPO slightly exceeds FiPla ($+2.9\%$), showing that the combination of immediate rewards and proper tuning is sufficient for PPO to match the myopic-optimal baseline. This suggests that the failure of RL on the delayed scenario stems from both credit-assignment difficulty and hyperparameter sensitivity, rather than an intrinsic inability to learn the task.

Table 4: PPO diagnostic on `survival-second-price-exponential` (η=50, 1 000 seeds; gain vs. FiPla of the same variant). *PPO (tuned)*: retrained on the immediate kernel with the same hyperparameters as the delayed scenario. *PPO (search)*: retrained with the best configuration from a 124-trial grid search; evaluated on the immediate kernel only.

	Delayed reward		Immediate reward (retrained)	
Policy	Reward	vs. FiPla	Reward	vs. FiPla
FiPla	1.706	—	1.830	—
PPO (tuned)	1.592	-6.7%	1.787	-2.4%
PPO (search)	—	—	1.883	$+2.9\%$

Q4 / Performance–cost tradeoff. *""Could the authors report a performance–cost tradeoff table, such as reward versus average decision-time latency for MPC with different H, M, and K? Even wall-clock time per episode or per bid on a standard CPU would be useful.""*

MPC(H=10, K=1) provides the best computation–performance trade-off among MPC variants (Table 5), improving the mean reward by $+31.6\%$ over FiPla with only 110.5 ms latency per bid. Increasing the horizon from H=10 to H=25 improves reward by an additional $+10.4\%$ relative to FiPla, but nearly doubles the inference latency. Increasing the number of rollout samples from K=1 to K=2 increases latency by approximately $6\times$ while providing negligible additional reward. Thus, longer planning improves performance, but the marginal gain quickly diminishes relative to the additional computation.

Compared with Deep RL (Table 6), MPC requires substantially less offline training computation: 0.08 CPU-hours for V_0 pretraining versus 3.2–40 CPU-hours for PPO/SAC/TD3. The trade-off is shifted to online inference: MPC performs explicit rollouts at decision time (110–835 ms per bid), whereas neural policies require only a single forward pass (< 0.31 ms).

Table 5: Performance–latency trade-off for MPC variants (M=10 Monte Carlo rollouts, survival scenario, 100 seeds, 1 CPU core, standard x86 server, no GPU). FiPla is the greedy myopic policy with access to the expected-reward oracle, used as the reference baseline. Shading is included for comparison. **Latency** is the mean wall-clock computation time per bidding decision. Relative latency is measured with respect to MPC(H=10, K=1). Evaluating MPC(H=25, K=2) with 1 000 seeds would require more than 40 hours; rewards are therefore from the same 100-seed runs.

Policy	Mean reward	Latency (ms/bid)	Relative latency
FiPla	1.606	0.001	—
Shading($\alpha = 0.7$)	1.673	24.1	—
MPC($H = 10, K = 1$)	2.113	110.5	1.0×
MPC($H = 25, K = 1$)	2.281	204.8	1.9×
MPC($H = 10, K = 2$)	2.128	657.6	6.0×
MPC($H = 25, K = 2$)	2.278	834.8	7.6×

Relative latency is defined as $\tau(\pi)/\tau(\text{MPC}(H = 10, K = 1))$, where $\tau(\pi)$ denotes the average computation time per bid.

Table 6: Offline training and online inference cost comparison between Deep RL and MPC on the survival scenario ($\eta = 50$, 100 seeds, 4 CPU cores, standard x86 server, no GPU). FiPla is the greedy myopic policy with access to the expected-reward oracle. **Wall (h)** denotes wall-clock training time. **CPU-h** denotes the total consumed CPU core-hours (RL uses 4 cores through vectorised environments; MPC V_0 pretraining uses 1 core). **Infer. (ms)** is the mean wall-clock computation time per bidding decision. The performance ratio is $\rho_\pi = \bar{R}(\pi)/\bar{R}(\text{FiPla})$, and the reward gain is $(\rho_\pi - 1) \times 100\%$. Inference efficiency is defined as $\mathcal{E}_{\text{infer}}(\pi) = (\bar{R}(\pi) - \bar{R}(\text{FiPla}))/\tau(\pi)$, where $\tau(\pi)$ is the inference latency. Higher values indicate more reward improvement per millisecond of computation; negative values indicate methods that consume computation while reducing performance relative to FiPla.

Policy	Type	Wall (h)	CPU-h	Infer. (ms)	ρ_π	Reward gain	$\mathcal{E}_{\text{infer}}$
FiPla	myopic	0.00	0.00	0.001	1.000	—	—
PPO	Deep RL	0.80	3.20	0.234	0.976	−2.4%	−0.164
SAC	Deep RL	10.00	40.00	0.306	0.483	−51.7%	−3.670
TD3	Deep RL	9.10	36.40	0.231	0.694	−30.6%	−2.260
MPC($H = 10, K = 1$)	MPC	0.08	0.08	110.5	1.316	+31.6%	4.59×10^{-3}
MPC($H = 25, K = 1$)	MPC	0.08	0.08	204.8	1.420	+42.0%	3.30×10^{-3}
MPC($H = 10, K = 2$)	MPC	0.08	0.08	657.6	1.325	+32.5%	7.93×10^{-4}
MPC($H = 25, K = 2$)	MPC	0.08	0.08	834.8	1.418	+41.8%	8.05×10^{-4}

All methods are evaluated on **100 seeds**. RL training uses 10M environment steps with four parallel CPU workers. MPC requires only the offline V_0 head pretraining ($\approx$ 5 minutes on one CPU core), resulting in approximately 40–500× lower CPU consumption than Deep RL. The computational trade-off is reversed during inference: MPC performs online Monte Carlo rollouts (110–835 ms per bid), whereas RL executes a single neural-network forward pass (< 0.31 ms).